

SDR SDRAM Technical Report

1. WHAT IS SDR SDRAM? - COMPREHENSIVE EXPLANATION

1.1 Definition and Terminology

SDR SDRAM = Single Data Rate Synchronous Dynamic Random Access Memory

Breaking down each term:

Single Data Rate (SDR):

- Transfers data ONCE per clock cycle
- Data transferred only on rising edge of clock
- Contrast with DDR (Double Data Rate) which transfers on both rising and falling edges
- At 133 MHz: 133 million data transfers per second

Synchronous:

- All operations synchronized to external clock signal (CLK)
- Commands sampled on clock rising edge
- Predictable timing based on clock cycles
- Contrast with asynchronous DRAM where timing based on signal propagation delays

Dynamic:

- Memory cell uses capacitor that leaks charge over time
- Requires periodic refresh to maintain data
- "Dynamic" because stored charge is not static
- Contrast with "Static" RAM (SRAM) which holds data indefinitely

Random Access Memory:

- Any memory location accessed in equal time
- No need to access sequentially
- Direct addressing to any byte/word

1.2 Historical Context

Evolution Timeline:

- **1970s-1990s:** Asynchronous DRAM (FPM, EDO)
- **1992:** Samsung introduces first SDR SDRAM (KM48SL2000, 16Mbit)
- **1993:** JEDEC standardizes SDR SDRAM
- **1997:** DDR SDRAM demonstrated
- **2000:** SDR SDRAM dominates PC market
- **2003-2005:** DDR begins replacing SDR
- **Today:** Legacy systems, educational use, cost-sensitive embedded applications

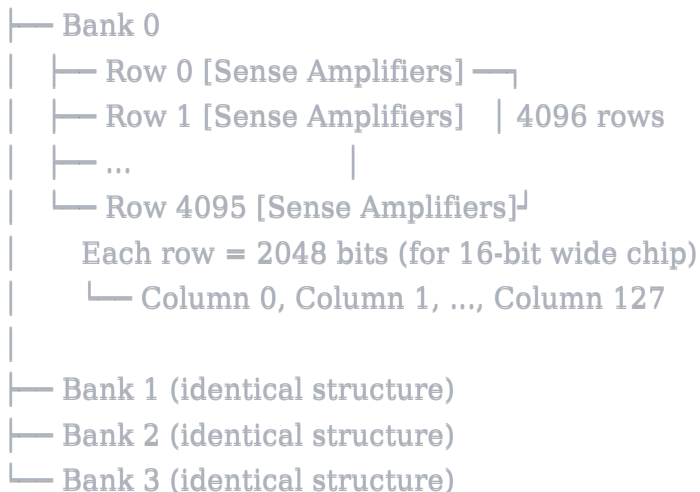
Why it was revolutionary:

- Asynchronous DRAM required complex timing with multiple RAS/CAS edges
- SDR SDRAM simplified interface: one command per clock cycle
- Enabled pipelined operations and burst transfers
- Made memory controllers more predictable and efficient

1.3 Core Architecture - Deep Dive

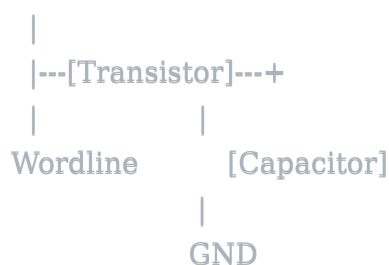
Physical Structure:

SDRAM Chip



Memory Cell - The 1T1C Building Block:

Bitline



- **Transistor:** Acts as switch, controlled by wordline
- **Capacitor:** Stores charge (charged = 1, discharged = 0)
- **Typical capacitance:** 20-30 femtofarads (fF)
- **Charge retention:** ~64ms before refresh needed

How a Cell Works:

1. Write Operation:

- Wordline activated (transistor ON)
- Bitline driven high (write 1) or low (write 0)
- Capacitor charges or discharges through transistor
- Wordline deactivated (transistor OFF, charge trapped)

2. Read Operation:

- Capacitor initially holds charge (or doesn't)
- Wordline activated
- Charge (or lack of) flows to bitline
- Sense amplifier detects tiny voltage change
- Sense amplifier drives bitline to full voltage
- **CRITICAL:** Read is destructive! Original charge dissipated
- Sense amplifier automatically writes back to restore cell

3. Refresh Operation:

- Periodically activate each row
- Sense amplifiers read and restore charge
- Must happen every 64ms (typical)
- Prevents data loss from capacitor leakage

1.4 Why "Synchronous" Matters

Asynchronous DRAM (Old Way):

1. Assert RAS# (start row access) → wait tRAS (fixed time)
2. Assert CAS# (start column access) → wait tCAS (fixed time)
3. Data appears on bus → read it quickly before it disappears
4. Timing based on signal propagation delays (nanoseconds)
5. Controller must track absolute time, not clock cycles

Synchronous SDRAM (New Way):

Clock Cycle 0: Send ACTIVATE command (RAS#, CAS#, WE# combination)
Clock Cycle 1: NOP (no operation)
Clock Cycle 2: NOP
Clock Cycle 3: Send READ command
Clock Cycle 4: NOP
Clock Cycle 5: NOP (if CAS Latency = 3)
Clock Cycle 6: Data appears on bus
Clock Cycle 7: Next data word (if burst mode)

Benefits of Synchronous Operation:

- **Predictable:** Everything happens on clock edges
- **Pipelined:** Send new command before previous completes
- **Simplified Controller:** Count cycles, not nanoseconds
- **Higher Frequency:** Clock can run faster than async signal propagation
- **Burst Mode:** Sequential accesses automatic after one command

1.5 The "Single Data Rate" Concept

Data Transfer Points:



SDR Characteristics:

- 1 data word per clock cycle maximum
- Data sampled/driven on rising edge only
- Falling edge unused for data transfer
- At 133 MHz: 133 million words/second
- With 64-bit bus: 1066 MB/s (133×8 bytes)

Why Not Always Transfer Data?

- Commands also need clock cycles (ACTIVATE, PRECHARGE)
- Wait cycles for internal timing (tRCD, tRP)
- Refresh cycles periodically interrupt
- Real efficiency: 70-90% depending on access patterns

1.6 Memory Organization - Why Banks, Rows, Columns?

Why Not Simple Linear Array?

Traditional SRAM view:

```
Address 0x0000 → Byte 0
Address 0x0001 → Byte 1
...
Address 0xFFFF → Byte 65535
```

SDRAM reality:

```
Address = [Bank Bits][Row Bits][Column Bits]
Example: 0b_01_0000110010111_00101101
      |||      ||
      |||      ||
      |||      || └─ Column 45
      |||      || └─ Row 395
      || └────── Row address
      || └────── Bank 1
      || └────── Bank address
```

Why This Organization?

1. **Manufacturing Reality:**

- DRAM cells arranged in 2D array for practical layout
- Row/column selection minimizes wiring complexity
- One row decoder + one column decoder = efficient

2. **Sensing Physics:**

- Entire row must be read to sense amplifiers
- Sense amplifiers compare cell voltage to reference
- Row remains "open" in sense amplifiers after read
- Accessing same row again is fast (data already in sense amps)
- Accessing different row requires precharge → activate cycle

3. **Multiple Banks = Parallelism:**

- Each bank operates independently
- While Bank 0 serves data, Bank 1 can precharge/activate
- Hides latency through interleaving
- Critical for achieving high bandwidth

Example: MT48LC4M16A2 (64Mbit chip)

Total: 64 Megabits = 4 Meg × 16-bit words

Organization:

- 4 banks (BA[1:0] = 2 bits)
- 4096 rows per bank (A[11:0] = 12 bits)
- 256 columns per row (A[7:0] = 8 bits)
- 16-bit data width (DQ[15:0])

Address breakdown:

BA[1:0]: 00, 01, 10, 11 (4 banks)

A[11:0]: 0-4095 (row address)

A[7:0]: 0-255 (column address)

Each bank: 4096 rows × 256 columns × 16 bits = 16 Megabits

Total: 4 banks × 16 Mb = 64 Megabits

1.7 What Makes SDR SDRAM "Special"

Compared to SRAM:

- 4-16x higher density (1T1C vs 6T)
- 10-20x cheaper per bit
- But requires refresh and controller

Compared to Async DRAM:

- Clock-synchronous = easier controller design
- Pipelining = higher throughput
- Burst mode = efficient sequential access
- Multiple banks = latency hiding

Compared to DDR:

- Simpler interface (single edge)
- Lower power (less switching)
- Cheaper manufacturing (less complex I/O)
- But half the bandwidth at same clock frequency

The Sweet Spot: SDR SDRAM hit the perfect balance for late 1990s computing:

- Fast enough for Pentium II/III processors
- Cheap enough for mainstream PCs
- Simple enough for cost-effective controllers
- Dense enough for 32-512 MB modules

1.8 Key Concepts Summary

What SDR SDRAM IS:

- Clock-synchronous DRAM with single-edge data transfer
- Command-based operation (not async signal timing)
- Bank/row/column hierarchical organization
- Requires periodic refresh (every 64ms typical)
- Supports burst transfers and pipelining

What SDR SDRAM IS NOT:

- Not SRAM (totally different technology)
- Not asynchronous (clock-synchronized)
- Not DDR (single data rate, not double)
- Not directly addressable (needs controller)
- Not static (requires refresh)

Critical Technical Points:

- 1T1C cell = cheap + dense + needs refresh
- Banks = independent operation = parallelism
- Rows = activated to sense amplifiers = page mode
- Columns = selected from active row = fast access
- Clock = synchronizes everything = predictable timing

1.2 Key Operations

- RAS# command activates row in specified bank
- Row data transferred to sense amplifiers
- Sense amplifiers act as latches holding row data
- tRCD delay required before column access

COLUMN ACCESS (Read/Write)

- Returns sense amplifiers to idle state
- Required before activating different row in same bank
- tRP delay must elapse before next activation
- Can be explicit command or automatic after read/write
- Capacitors leak charge over time - data must be restored
- AUTO REFRESH: controller issues command, internal counter handles rows
- Typical: 4096 refresh cycles every 64ms (15.625 μ s per cycle)
- All banks must be precharged before refresh

2.3 Timing Parameters

- **tRCD:** RAS to CAS delay (row activation to column access)
- **tRP:** Row precharge time
- **tRAS:** Minimum row active time
- **tRFC:** Auto refresh cycle time
- **CL (CAS Latency):** Clock cycles from read command to data output (typically 2-3 cycles for SDR)

2.4 Command Structure

Commands defined by CS#, RAS#, CAS#, WE# pin combinations:

- ACTIVE: Opens row (RAS=L, CAS=H, WE=H)
- READ/WRITE: Column access (RAS=H, CAS=L, WE varies)
- PRECHARGE: Closes row (RAS=L, CAS=H, WE=L)
- AUTO REFRESH: Refresh cycle (RAS=L, CAS=L, WE=H)
- LOAD MODE REGISTER: Configure parameters (RAS=L, CAS=L, WE=L)

3. DIFFERENCES FROM OTHER RAM TYPES

3.1 vs SRAM

Feature	SDR SDRAM	SRAM
Cell Structure	1T+1C (transistor+capacitor)	4-6 transistors
Density	High (4-16x SRAM)	Low
Speed	Slower (CL latency, refresh overhead)	Faster (immediate access)
Cost/Bit	Much cheaper	Expensive
Refresh	Required (every 64ms)	Not required
Complexity	Controller needed	Direct access
Interface	Command-based, clocked	Asynchronous
Power	Lower standby, refresh current	Higher

3.2 vs Asynchronous DRAM

- **Synchronous:** Clock-driven state machine vs direct signal response
- **Pipelining:** Can accept new command before previous completes
- **Burst Mode:** Automatic sequential/interleaved access vs per-access addressing
- **Interleaving:** Multiple independent banks enable concurrent operations
- **Performance:** Higher bandwidth through pipelining and burst access

3.3 vs DDR SDRAM

Feature	SDR	DDR
Data Transfer	Single edge (rising only)	Double edge (rising + falling)
Prefetch	1n (single word)	2n (two words)
Voltage	3.3V	2.5V
Bandwidth	1 word/cycle	2 words/cycle
Clock Rate	66-200 MHz	Higher effective rate

3.4 vs Other DRAM Types

- **RDRAM:** Proprietary, narrow bus (16-bit vs 64-bit), higher latency, licensing costs
- **SLDRAM:** Open standard, 11-bit command bus, different clocking - never mass-produced
- **SGRAM:** Graphics-specific with block write/bit masking, single-port but simulates dual-port
- **GDDR:** Graphics DDR with higher clock frequencies, optimized for GPU bandwidth

4. WHY USE SDR SDRAM

4.1 Technical Advantages

Cost-Effective

- Significantly cheaper per bit than SRAM
- Simple 1T+1C cell = higher density = lower manufacturing cost

Sufficient Speed

- PC66: 533 MB/s ($66 \text{ MHz} \times 64\text{-bit} / 8$)
- PC100: 800 MB/s ($100 \text{ MHz} \times 64\text{-bit} / 8$)
- PC133: 1066 MB/s ($133 \text{ MHz} \times 64\text{-bit} / 8$)
- Adequate for many embedded systems and legacy applications

High Density

- Typical: 64Mb-512Mb per chip in single package
- Example: MT48LC4M16A2 = 64Mb ($4\text{M} \times 16\text{-bit}$)

Standardized

- JEDEC standards ensure interoperability
- Well-documented timing and commands
- Multiple vendors (Samsung, Micron, Hynix)

4.2 Use Cases

FPGA Projects

- Limited internal memory (distributed RAM, block RAM, UltraRAM insufficient)
- Need large buffers/frame buffers beyond on-chip capacity
- Cost-sensitive designs

Legacy Systems

- Late 1990s/early 2000s PC systems
- Embedded systems still in production
- Replacement/maintenance of existing hardware

Educational Purposes

- Foundation for understanding DDR technologies
- Controller design teaches memory timing concepts
- Simpler than DDR for learning

Specific Applications

- Video frame buffers
- Audio buffering
- Data acquisition systems
- Network packet buffers
- Industrial control systems

4.3 Technical Trade-offs

When to Use SDR SDRAM:

- Cost is primary concern
- Moderate speed requirements (< 1 GB/s)
- Simple controller acceptable
- Power budget allows refresh current

When NOT to Use:

- Need highest performance (use DDR3/DDR4/DDR5)
- Ultra-low latency critical (use SRAM)
- Battery-powered with strict power limits (use LPDDR)
- High-end graphics (use GDDR)

5. CONTROLLER IMPLEMENTATION

5.1 Initialization Sequence

1. Wait 100 μ s with CKE HIGH
2. PRECHARGE ALL banks
3. Wait tRP time
4. Issue 2 $\times$ AUTO REFRESH commands (wait tRFC between)
5. LOAD MODE REGISTER (burst length, CAS latency, etc.)
6. Wait tMRD time
7. Ready for operation

5.2 FSM States

- **INIT:** Power-up initialization sequence
- **IDLE:** Ready for refresh or read/write
- **REFRESH:** Perform auto refresh cycle
- **ACTIVATE:** Open row in bank
- **READ/WRITE:** Column access operations
- **PRECHARGE:** Close row
- **WAIT:** NOP cycles for timing compliance

5.3 Clock Considerations

- **Clock Forwarding:** ODDR2 primitive to output clock to SDRAM
- **Phase Shift:** IODELAY2 for setup/hold time margin
- **IOB Packing:** Registers in I/O blocks minimize skew
- Typical: 100 MHz system clock → 10ns period

5.4 Critical Design Points

- Refresh every 15.625 μ s maximum (for 64ms/4096 rows)
- All timing parameters must be met (tRCD, tRP, tRAS, tRFC)
- NOP commands during wait cycles
- Handle three scenarios: row open, no row open, wrong row open
- Mode register: burst length, burst type, CAS latency

6. MODE REGISTER

6.1 Configuration Parameters

- **Burst Length:** 1, 2, 4, 8 words, or full-row
- **Burst Type:** Sequential (0-1-2-3) or Interleaved (0-1-3-2 for BL=4)
- **CAS Latency:** 2 or 3 cycles (device/frequency dependent)
- **Operating Mode:** Normal operation
- **Write Burst Mode:** Match read burst or single-location only

7. PHYSICAL SPECIFICATIONS

7.1 Example: MT48LC4M16A2

- Capacity: 64Mb ($1\text{M} \times 16 \times 4$ banks)
- Organization: 4 banks, 4096 rows, 256 columns
- Interface: 13-bit row address, 8-bit column address, 2-bit bank address
- Data width: 16-bit (with DQML/DQMH byte masking)
- Voltage: 3.3V
- Form factor: 168-pin DIMM or 144-pin SO-DIMM

7.2 Pin Configuration

- CLK, CKE: Clock and clock enable
- CS#: Chip select
- RAS#, CAS#, WE#: Command inputs
- A[12:0]: Address bus
- BA[1:0]: Bank address
- DQ[15:0]: Bidirectional data bus
- DQML, DQMH: Data mask signals

8. PERFORMANCE CHARACTERISTICS

8.1 Latency

- Row access: $\sim 50\text{ns}$ (DRAM array sensing)
- Column access: $< 10\text{ns}$ (from open row)
- CAS Latency: 10-15ns (2-3 cycles at 133 MHz)
- Not inherently faster than burst EDO DRAM, but higher bandwidth through interleaving

8.2 Bandwidth Calculation

Example PC133:

- Clock: 133 MHz
- Bus width: 64-bit
- Bandwidth: $133\text{ MHz} \times 64\text{ bits} \div 8 = 1066\text{ MB/s}$

8.3 Efficiency Factors

- Bank interleaving improves utilization
- Burst transfers reduce addressing overhead
- Pipelining allows command overlap
- Refresh overhead (~1-2% bandwidth loss)

9. BANK INTERLEAVING - DETAILED EXPLANATION

9.1 What is Bank Interleaving?

Bank interleaving is a technique where memory accesses are distributed across multiple independent banks to hide precharge/activate latency and maximize bandwidth utilization.

Without Interleaving (Single Bank):

Access sequence: ACTIVATE → READ → PRECHARGE → wait → ACTIVATE → READ...
 Problem: Bus idle during precharge (tRP) + activate (tRCD) delays

With Interleaving (Multiple Banks):

Bank 0: ACTIVATE → READ (while Bank 1 precharging/activating)
 Bank 1: ACTIVATE → READ (while Bank 2 precharging/activating)
 Bank 2: ACTIVATE → READ (while Bank 3 precharging/activating)
 Bank 3: ACTIVATE → READ (while Bank 0 precharging/activating)
 Result: Continuous data flow, latencies hidden by parallel bank operations

9.2 How Interleaving Works

Address Mapping: Memory controller maps sequential addresses to different banks using address bit partitioning.

Common interleaving schemes:

- **Chip-Bank-Row-Col:** Non-interleaved, random traffic optimization
- **Chip-Row-Bank-Col:** Bank interleaving, sequential traffic optimization
- **Row-Chip-Bank-Col:** Bank + chip-select interleaving, multi-chip systems

Example 4-Bank Interleaving:

Clock 1: Controller sends address to Bank 0 → ACTIVATE
Clock 2: Controller sends address to Bank 1 → ACTIVATE
Receives data from Bank 0 (Bank 0 in precharge)
Clock 3: Controller sends address to Bank 2 → ACTIVATE
Receives data from Bank 1 (Bank 1 in precharge)
Clock 4: Controller sends address to Bank 3 → ACTIVATE
Receives data from Bank 2 (Bank 2 in precharge)
Clock 5: Back to Bank 0 (now ready after completing tRC cycle)
Receives data from Bank 3

9.3 Performance Benefits

Latency Hiding:

- While one bank is serving data, other banks undergo precharge/activate
- tRP (precharge time) + tRCD (activate time) completely masked
- Can hide entire tRC (row cycle time) with sufficient banks

Bandwidth Improvement:

- Theoretical best case: 100% efficiency with 8 banks
- Typical improvement: 30-70% bandwidth increase vs non-interleaved
- Depends on access patterns and controller efficiency

Optimal Conditions:

- Sequential memory access patterns
- 4-8 banks available
- Controller with intelligent address mapping
- Sufficient queue depth for command reordering

9.4 Interleaving Requirements

Hardware:

- Multiple independent banks (2, 4, or 8 in SDR SDRAM)
- Memory controller supporting bank interleaving logic
- Proper address decoding for bank distribution

Access Pattern:

- Best: Sequential or near-sequential (video buffers, DMA transfers)
- Good: Working set spans multiple banks
- Poor: Random access concentrated in single bank

9.5 Bank vs Rank vs Channel Interleaving

Type	Description	Benefit
Bank Interleaving	Use multiple banks within single chip	Hide tRC latency
Rank Interleaving	Use multiple ranks (chip groups)	Increase total bandwidth
Channel Interleaving	Use multiple memory channels	Double/quadruple bandwidth

SDR SDRAM primarily uses **bank interleaving** (2-4 banks typical).

9.6 Practical Example

Scenario: 4-bank SDRAM, PC133, tRCD=3 cycles, tRP=3 cycles

Without interleaving:

ACTIVATE → wait 3 → READ → PRECHARGE → wait 3 → ACTIVATE → wait 3 → READ
 Total: 10 cycles for 2 data words = 5 cycles/word

With 4-bank interleaving:

Cycle 1: ACTIVATE Bank 0
 Cycle 4: READ Bank 0, ACTIVATE Bank 1
 Cycle 5: READ Bank 0, ACTIVATE Bank 2
 Cycle 6: READ Bank 0, ACTIVATE Bank 3
 Cycle 7: READ Bank 1, ACTIVATE Bank 0 (recycled)
 Cycle 8: READ Bank 2
 Continuous: 1 data word per cycle after initial latency

Result: 5x throughput improvement for sequential access

9.7 Limitations

Not Always Beneficial:

- Random access patterns may not align with bank boundaries
- Small data structures may reside in single bank
- Overhead of controller complexity
- Refresh cycles interrupt interleaving flow

Bank Conflicts:

- Multiple accesses to same bank force serialization
- Cache line conflicts at higher software layers propagate to memory
- Address mapping must distribute access evenly

Diminishing Returns:

- Going from 2→4 banks: significant improvement
- Going from 4→8 banks: moderate improvement
- Beyond 8 banks: minimal additional benefit for SDR SDRAM

10. ADVANCED TOPICS

10.1 Burst Modes Explained

Sequential Burst (M3=0): Starting address 5, burst length 4: accesses 5→6→7→4 (wraps at boundary) Starting address 5, burst length 8: accesses 5→6→7→0→1→2→3→4

Interleaved Burst (M3=1): Starting address 5, burst length 4: accesses 5→4→7→6 Starting address 5, burst length 8: accesses 5→4→7→6→1→0→3→2 Uses XOR between counter and address for ordering

Critical Word First: Allows CPU to receive requested word first, then rest of cache line Most useful with sequential burst mode

10.2 Page Mode vs Closed Page

Page Mode (Open Page Policy):

- Keep row open after access
- Benefit: Fast successive accesses to same row
- Drawback: Higher power, potential bank conflicts

Closed Page (Auto-Precharge):

- Close row immediately after access
- Benefit: Next access can be to any row
- Drawback: Additional tRP + tRCD latency

Controllers dynamically choose based on access patterns.

10.3 Command Pipelining

SDRAM allows overlapping commands:

Clock 0: ACTIVATE Bank 0
 Clock 1: ACTIVATE Bank 1 (while Bank 0 activating)
 Clock 2: ACTIVATE Bank 2 (Bank 0 ready for READ)
 Clock 3: READ Bank 0, ACTIVATE Bank 3
 Clock 4: READ Bank 0, READ Bank 1

This pipelining is key to achieving high throughput.

10.4 Refresh Strategies

Distributed Refresh:

- Issue AUTO REFRESH every 15.625µs (for 64ms/4096 rows)
- Minimal impact on bandwidth (~1.5%)
- Predictable latency

Burst Refresh:

- Issue 4096 refresh commands consecutively
- Once every 64ms
- Frees up bandwidth between refresh bursts
- Large latency spike during burst

10.5 DQM (Data Mask) Usage

During Reads:

- DQM high 2 cycles before read → suppress output
- Allows safe transition between read/write on DQ bus

During Writes:

- DQM high during write → mask those bytes
- Enables byte-level write control on word-wide interface
- DQML masks low byte (DQ[7:0]), DQMH masks high byte (DQ[15:8])

11. DESIGN CONSIDERATIONS FOR FPGA IMPLEMENTATION

11.1 Clock Domain Crossing

- SDRAM operates synchronously with system clock
- Use same clock for FPGA logic and SDRAM when possible
- Clock forwarding ensures timing alignment

11.2 IOB Register Placement

Critical for meeting timing:

- All output signals must use IOB flip-flops
- Minimize skew between CLK and control/data signals
- Use constraints: `(* IOB = "TRUE" *)`

11.3 Clock Forwarding Techniques

ODDR2 Primitive:

- Forwards system clock to SDRAM
- D0=0, D1=1 creates clock output on both edges
- Provides half-cycle setup/hold margin

IODELAY2 Primitive:

- Fine-tunes clock phase
- Compensates for PCB trace delays
- Typical setting: 50-150 taps (device dependent)

11.4 Simulation with Vendor Models

- Use SDRAM behavioral models (Micron, Samsung)
- Verify initialization sequence timing
- Check tRCD, tRP, tRAS, tRC compliance
- Validate refresh timing

11.5 Common Pitfalls

- Forgetting refresh cycles (data corruption)
- Violating tRAS minimum (precharge too early)
- Missing NOP cycles during wait states
- Incorrect mode register programming
- Clock skew between FPGA and SDRAM